```
chain(turtle)
.forward(2)
.rotate(90)
.forward(4)
```

Before CoffeeScript 1.7

```
chain turtle
.forward 2
.rotate 90
.forward 4
```

CoffeeScript 1.7 and later

To do this you need to create a `chain` function that takes an object and returns a fluent interface of the object's methods. This is one of those cases where you need Agtron's help. The implementation of `chain` that he helps you create and an example of using it are shown in the following listing. If you don't understand exactly how it works, then it's okay to move on and come back to it later.

Listing 7.6. The `chain` function

```
chain = (receiver) ->
  wrapper = Object.create receiver
  for key, value of wrapper
    if value?.call
      do ->
        proxied = value
        wrapper[key] = (args...) ->
          proxied.call receiver, args...
          wrapper
  wrapper

turtle =
  forward: (distance) ->
    console.log "moving forward by #{distance}"
  rotate: (degrees) ->
    console.log "rotating #{degrees} degrees"

chain(turtle)
.forward(5)
.rotate(90)
```

- Inherit from the receiving object that provides the API.
- Comprehend the properties of the wrapper object, all of which come from the prototype.
- Bind the local variable to the function for use later.
- Create a closure to bind the local variable.
- Duck typing: if the value of the property has a call method, then treat it as a function.
- Define a property on the wrapper that's a function.
- Fluent interface: return the wrapper so that calls can be chained.
- This function calls the proxied function with the argument and with the receiver bound.
- Fluent interface: return the wrapper from the chain function so that the chain can be started.
- The chained method calls.

By providing some syntactic sugar, CoffeeScript can make more-advanced JavaScript techniques a bit more manageable. This sugar doesn't come for free, though; sometimes it can make things ambiguous.

7.7. Ambiguity